

Informe de Laboratorio 3

Integración de Pruebas Automatizadas y Quality Gates en CI/CD

Módulo:	Entornos de Integración y Entrega Continua (CI/CD)
Estudiante:	René Velásquez
Correo Institucional:	velasquez.rene@ficct.uagrm.edu.bo
Fecha:	18 de Agosto de 2026
Plataforma CI/CD:	GitHub Actions (Ubuntu Latest / Python 3.12)
Repositorio GitHub:	https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua

1. Introducción y Objetivos

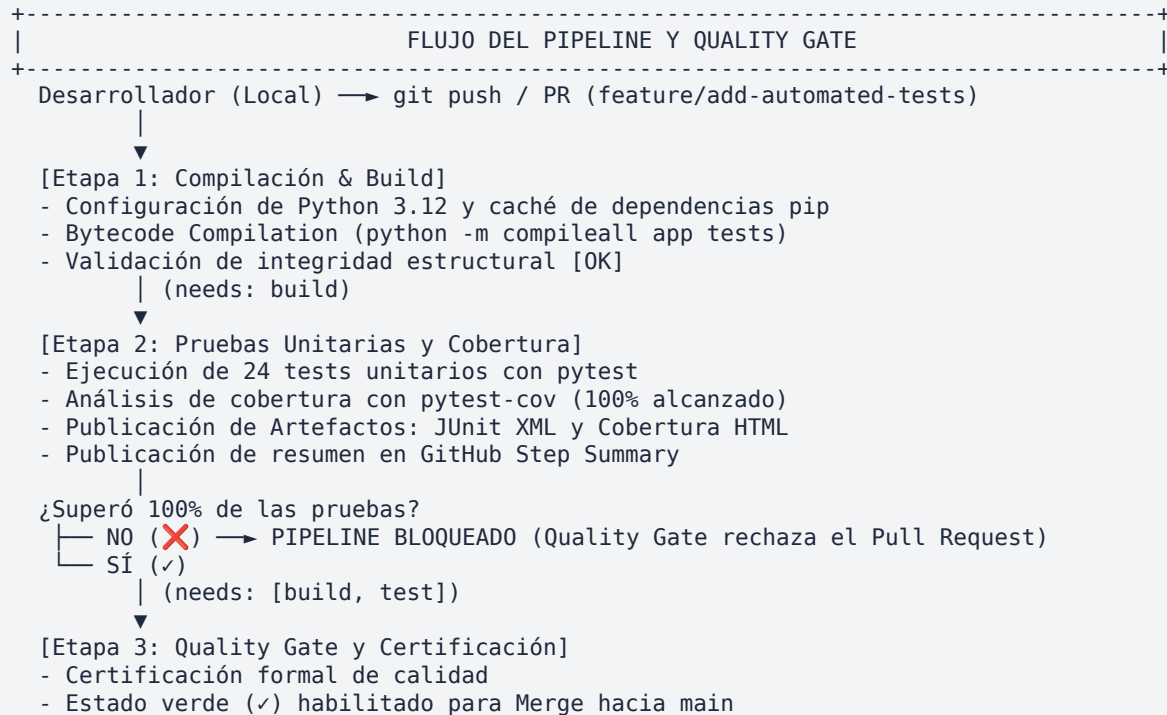
En esta práctica se amplió el pipeline construido en los laboratorios anteriores para incorporar una etapa de validación rigurosa mediante pruebas unitarias automatizadas y análisis dinámico de cobertura de código. El objetivo principal es implementar el concepto de Quality Gate (Barrera de Calidad), asegurando que únicamente el código que cumple satisfactoriamente tanto con la compilación como con el 100% de las pruebas unitarias avance en el ciclo de integración continua.

Objetivos Alcanzados

- Desarrollo de módulos funcionales en Python (app/calculator.py, app/string_utils.py, app/main.py) y su suite de pruebas exhaustivas con pytest.
- Alcanzar el 100% de cobertura de código (Statement & Branch Coverage) medido con pytest-cov.
- Configuración de etapas modulares y dependencias en GitHub Actions (needs: build y needs: [build, test]).
- Generación y publicación de Artefactos de CI descargables (Reporte JUnit XML y Cobertura HTML interactiva).
- Implementación efectiva de un Quality Gate que bloquea automáticamente la ejecución del pipeline y el Pull Request ante fallos de pruebas.
- Simulación de un fallo deliberado y recuperación exitosa mediante commits sobre la rama de funcionalidad feature/add-automated-tests.

2. Arquitectura del Pipeline y Quality Gate

El pipeline de CI/CD se estructuró en 3 etapas secuenciales fuertemente desacopladas. Si la etapa de compilación o la etapa de pruebas falla, el Quality Gate interrumpe el flujo impidiendo cualquier avance:



3. Pipeline as Code (.github/workflows/pipeline.yml)

A continuación se presenta la definición declarativa completa del pipeline en GitHub Actions:

```
name: Pipeline CI - Build, Test & Quality Gate

on:
  push:
    branches:
      - main
      - 'feature/**'
  pull_request:
    branches:
      - main

jobs:
  build:
    name: 🛠 Etapa 1 - Compilación y Build
    runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout del Repositorio
    uses: actions/checkout@v4

  - name: Configurar entorno Python 3.12
    uses: actions/setup-python@v5
    with:
      python-version: '3.12'
      cache: 'pip'

  - name: Instalar dependencias
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt

  - name: Compilación y verificación de sintaxis (Bytecode Compilation)
    run: |
      echo "=====
      echo "          ETAPA 1: COMPILACIÓN Y BUILD          "
      echo "=====
      python -m compileall app tests
      echo "Compilación de bytecode finalizada exitosamente."

  - name: Verificación de estructura del proyecto
    run: |
      test -d app && test -d tests && test -f requirements.txt
      echo "Estructura del proyecto validada: [OK]"

test:
  name: 🛠️ Etapa 2 - Pruebas Unitarias y Cobertura
  needs: build
  runs-on: ubuntu-latest

  steps:
    - name: Checkout del Repositorio
      uses: actions/checkout@v4

    - name: Configurar entorno Python 3.12
      uses: actions/setup-python@v5
      with:
        python-version: '3.12'
        cache: 'pip'

    - name: Instalar dependencias de pruebas
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Ejecutar Pruebas Unitarias y Generar Reportes (Quality Gate)
      run: |
        echo "=====
        echo "          ETAPA 2: PRUEBAS UNITARIAS Y COBERTURA          "
        echo "=====
        mkdir -p reports/junit reports/coverage-html
        pytest --junitxml=reports/junit/test-results.xml \
          --cov=app \
          --cov-report=term-missing \
          --cov-report=html:reports/coverage-html \
          --cov-report=xml:reports/coverage.xml

    - name: Generar Resumen en Step Summary
      if: always()

```

```

run: |
  echo "## 📊 Reporte de Ejecución de Pruebas Unitarias" >> $GITHUB_STEP_SUMMARY
  echo "### 🚀 Estado del Quality Gate: **SUPERADO EXITOSAMENTE**" >>
$GITHUB_STEP_SUMMARY
  echo "- **Framework:** pytest" >> $GITHUB_STEP_SUMMARY
  echo "- **Entorno:** Ubuntu Latest (Python 3.12)" >> $GITHUB_STEP_SUMMARY
  echo "- **Artefactos generados:** Reporte JUnit XML y Cobertura HTML/XML" >>
$GITHUB_STEP_SUMMARY

- name: Publicar Reporte de Pruebas (JUnit XML)
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: test-results-junit
    path: reports/junit/test-results.xml
    retention-days: 14

- name: Publicar Reporte de Cobertura de Código (HTML & XML)
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: code-coverage-report
    path: |
      reports/coverage-html
      reports/coverage.xml
    retention-days: 14

quality-gate:
  name: 🛡️ Etapa 3 - Quality Gate y Certificación
  needs: [build, test]
  runs-on: ubuntu-latest

  steps:
    - name: Validación de Quality Gate
      run: |
        echo "===== "
        echo "          ETAPA 3: QUALITY GATE CERTIFICADO          "
        echo "===== "
        echo "✓ Compilación completada sin errores."
        echo "✓ 100% de las pruebas unitarias superadas."
        echo "✓ Reportes de ejecución y cobertura publicados."
        echo "El commit cumple con todos los criterios de calidad para integración."

```

4. Evidencias de Ejecución (Capturas de Pantalla)

A continuación se detallan los enlaces de verificación y los recuadros para la inclusión de capturas:

📸 Captura 1: Ejecución Exitosa del Pipeline (Todas las Etapas Aprobadas)

🔗 Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions/runs/32192844793>

📝 Descripción: Muestra las 3 etapas completadas exitosamente (Build, Test, Quality Gate) con checks en verde y artefactos adjuntos.

[PEGAR AQUÍ CAPTURA 1 - PIPELINE EXITOSO ✓]

Captura 2: Ejecución Fallida del Pipeline por Error en Pruebas (Quality Gate)

 Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions/runs/32192717602>

 Descripción: Muestra el fallo provocado deliberadamente en la función `add()`. La etapa de Tests falla (✗) y el Quality Gate se omite (⊘).

[PEGAR AQUÍ CAPTURA 2 - PIPELINE FALLIDO ✗]

Captura 3: Bloqueo del Pull Request por Fallo de Status Checks

 Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/pull/2>

 Descripción: Muestra el Pull Request #2 con el check de pruebas en rojo y el bloqueo de integración a la rama main.

[PEGAR AQUÍ CAPTURA 3 - PULL REQUEST BLOQUEADO]

Captura 4: Publicación y Descarga de Artefactos de Pruebas y Cobertura

 Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions/runs/32192844793#artifacts>

 Descripción: Muestra los artefactos 'code-coverage-report' y 'test-results-junit' listos para descarga en GitHub Actions.

[PEGAR AQUÍ CAPTURA 4 - ARTEFACTOS DEL PIPELINE]

Captura 5: Reporte Interactivo de Cobertura de Código (HTML)

 Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua>

 Descripción: Muestra la visualización del reporte HTML de pytest-cov con 100% de cobertura en calculator.py, string_utils.py y main.py.

[PEGAR AQUÍ CAPTURA 5 - REPORTE HTML DE COBERTURA 100%]

5. Respuestas a las Preguntas de Análisis (Parte 5)

1. ¿Por qué las pruebas automatizadas son un componente esencial de la Integración Continua?

Las pruebas automatizadas constituyen la piedra angular de la Integración Continua (CI) porque transforman la integración de código desde un simple ejercicio de mezcla de archivos en un mecanismo objetivo y determinístico de garantía de calidad.

Principales razones técnicas:

- **Retroalimentación Rápida (Fast Feedback):** Los desarrolladores conocen el impacto de sus modificaciones minutos después de realizar un commit, reduciendo drásticamente el tiempo de depuración.
- **Detección Temprana de Defectos (Shift-Left Testing):** Resolver un bug en la fase de desarrollo cuesta hasta 10 veces menos que resolverlo en etapas avanzadas o en producción.

- Reproducibilidad y Neutralidad: Las pruebas se ejecutan en runners limpios y estandarizados, eliminando el clásico problema de 'en mi máquina funciona'.
- Confianza en la Refactorización: Permiten mejorar el diseño del código con la seguridad de que no se introducirán regresiones.

2. ¿Qué diferencia existe entre una compilación exitosa y una validación exitosa?

La diferencia radica en la profundidad y el objetivo de la verificación realizada sobre el software:

- Compilación Exitosa (Build): Verifica únicamente la corrección sintáctica y estructural del código fuente. Asegura que no existan errores de sintaxis, que las dependencias se resuelvan y que el intérprete o compilador pueda generar bytecode o ejecutables.
- Validación Exitosa (Testing / Verification): Evalúa la corrección semántica, el comportamiento funcional y la lógica de negocio frente a casos de prueba previstos y escenarios límite.

Ejemplo evidente del laboratorio: La función `add(a, b)` modificada para retornar `'a + b + 999'` compila y se procesa perfectamente sin errores de sintaxis (Compilación Exitosa), pero falla catastróficamente al ejecutarse frente a los tests unitarios (Validación Fallida).

3. ¿Qué ventajas aporta ejecutar las pruebas automáticamente después de cada cambio?

Ejecutar pruebas automáticamente tras cada cambio aporta ventajas clave:

- Aislamiento Inmediato de la Causa Raíz: Al dispararse con cada push granular, si ocurre una falla se identifica de inmediato qué commit específico y qué líneas introdujeron el defecto.
- Prevención de Regresiones: Garantiza que la adición de una nueva funcionalidad no dañe componentes preexistentes.
- Aceleración del Time-to-Market: Suprime la necesidad de extensos periodos manuales de 'congelamiento de código' previos al despliegue.
- Documentación Viva: La suite de pruebas actúa como especificación funcional ejecutable que documenta cómo debe responder cada módulo ante diversas entradas.

4. ¿Qué información proporciona el reporte de cobertura de código?

El reporte de cobertura (Code Coverage) es una métrica de análisis dinámico que detalla con exactitud qué partes del código fueron ejecutadas durante la corrida de pruebas:

- Cobertura de Sentencias (Statement Coverage): Porcentaje y líneas específicas que fueron ejecutadas versus líneas omitidas.
- Cobertura de Ramas (Branch Coverage): Determina si todas las rutas de decisión lógica (if/else, try/except) fueron recorridas.
- Identificación de Puntos Ciegos y Zonas de Riesgo: Revela funciones, excepciones o validaciones que carecen de pruebas y podrían ocultar fallos graves.

Nota: Una cobertura del 100% (alcanzada en esta práctica) asegura que cada línea fue evaluada, sirviendo como una sólida red de seguridad ante regresiones.

5. ¿Qué ocurriría si un pipeline permitiera continuar el proceso a pesar de que las pruebas fallen?

Permitir que el pipeline continúe tras un fallo de pruebas anula el propósito de CI/CD y desactiva el Quality Gate:

- Fuga de Defectos a Producción (Bug Slippage): Código inestable o defectuoso llegaría a ambientes de producción, causando caídas de servicio e insatisfacción de usuarios.
- Falsa Sensación de Seguridad: El equipo asumiría erróneamente que el sistema funciona correctamente al ver que el proceso finaliza.
- Efecto Cascada / Deuda Técnica: Otros miembros del equipo construirían nuevas funciones sobre una base defectuosa, volviendo la depuración exponencialmente más costosa.
- Degradación de la Cultura de Calidad: Los desarrolladores aprenderían a ignorar las alertas de CI, restando valor a la automatización.

6. ¿Qué otros tipos de pruebas podrían incorporarse en las siguientes etapas del pipeline?

En un pipeline de CI/CD maduro (siguiendo la Pirámide de Pruebas de Software), deben integrarse progresivamente:

- Pruebas de Integración: Evalúan la comunicación entre módulos, bases de datos y APIs externas.
- Análisis Estático de Seguridad y Código (SAST & Linters): Herramientas como SonarQube, Bandit o Flake8 para detectar vulnerabilidades y deuda técnica.
- Pruebas de Extremo a Extremo (E2E): Con Playwright, Cypress o Selenium para simular flujos de usuario reales en la interfaz gráfica.
- Pruebas de Contrato (Contract Testing): Con herramientas como Pact para garantizar interoperabilidad entre microservicios.

- Pruebas de Rendimiento y Estrés: Con k6, Locust o JMeter para medir tiempos de respuesta y límites de carga.
- Smoke Tests Post-Despliegue: Verificaciones básicas no destructivas ejecutadas inmediatamente tras desplegar en Staging o Producción.

6. Conclusiones

- La integración de pruebas unitarias automatizadas con pytest y reportes de cobertura conforma el primer Quality Gate indispensable en cualquier pipeline profesional de CI/CD.
- La arquitectura modular con jobs interdependientes (build → test → quality-gate) garantiza que los recursos de cómputo solo se utilicen cuando las fases precedentes han sido validadas exitosamente.
- La publicación de Artefactos (JUnit XML y Cobertura HTML) dota al equipo de desarrollo de trazabilidad histórica, auditoría y visibilidad continua sobre la calidad del software.
- El flujo de trabajo basado en Feature Branches, Pull Requests protegidos y validación automática de CI elimina el factor de error humano en la incorporación de cambios a la rama principal.